DJAMA ERP

AUDIT MODULE

# Espaces Privés

Analyse complète — sécurité, architecture, RLS, intégrations, UX et roadmap

Septembre 2026 · Confidentiel

| 5 P0 critiques | 7 P1 manquants | 6 P2 partiels | 4 P3 UX |
|---|---|---|---|

## 1 — Résumé exécutif

Le module Espaces Privés est actuellement une **maquette fonctionnelle** : l'interface est propre, la création d'espaces fonctionne, mais l'architecture sous-jacente comporte **5 failles critiques** qui empêchent tout déploiement professionnel.

**Problèmes critiques (P0) — à corriger avant toute autre chose :**

① Pas d'**organization_id** sur private_spaces → isolation multi-tenant inexistante.

② Toutes les opérations (créer, supprimer, assigner) sont des **appels directs Supabase depuis le client** — aucun API layer.

③ La **RLS est chef-scoped** (user_id = auth.uid()) uniquement — un membre de l'organisation ne peut pas accéder à un espace même s'il y est invité.

④ Pas de table **space_members** — les membres sont liés par une FK simple sur team_members.space_id (un membre = un seul espace).

⑤ La page **/membre/login?code=...** référencée dans les liens d'invitation **n'existe pas**.

**Résultat attendu après correction complète :**

Un espace privé multi-tenant, sécurisé bout en bout, avec rôles, documents, chat canal dédié, tâches liées, journal d'activité et connexions aux modules DJAMA existants.

## 2 — Architecture actuelle

### 2.1 Fichiers existants

```
src/app/client/espaces/page.tsx — frontend (477 lignes)
supabase/migrations/20260719_private_spaces.sql — migration DB
(aucune route API dédiée) — ■ ABSENT
(aucune page /membre/login) — ■ ABSENT
```

### 2.2 Schéma DB réel

**Table private_spaces :** id, user_id (PAS organization_id), name, description, color, access_code, is_active, created_at, updated_at

**Colonnes ajoutées :** team_members.space_id, team_messages.space_id, team_tasks.space_id, team_meetings.space_id

**RLS :** ps_own — FOR ALL USING (user_id = auth.uid())

**■ MANQUANT : organization_id, space_members (rôles), space_files, space_activity_log**

### 2.3 Flux de données actuel

Frontend → supabase.from('private_spaces').insert() direct (pas d'API)

Assignation membre → supabase.from('team_members').update({space_id}) direct

Suppression → supabase.from('private_spaces').delete() direct

Lectures → supabase.from('private_spaces').select('*').eq('user_id', uid)

■ **Aucun serveur entre le client et la base de données.**

## 3 — Classification P0 / P1 / P2 / P3

| PRIO | PROBLÈME / FONCTIONNALITÉ | STATUT |
|---|---|---|
| P0 | Pas d'organization_id sur private_spaces — isolation multi-tenant absente | ■ ABSENT |
| P0 | Toutes les opérations sont des appels client directs (pas d'API layer) | ■ ABSENT |
| P0 | RLS chef-scoped uniquement — membres invités ne peuvent pas lire leurs espaces | ■ ABSENT |
| P0 | Pas de table space_members — un membre ne peut appartenir qu'à un seul espace | ■ ABSENT |
| P0 | Page /membre/login?code=... inexistante — les liens d'invitation sont brisés | ■ ABSENT |
| P1 | Rôles dans un espace : owner / admin / member / viewer — absent | ■ ABSENT |
| P1 | Documents/fichiers dans un espace — absent (pas de bucket dédié) | ■ ABSENT |
| P1 | Journal d'activité de l'espace — absent | ■ ABSENT |
| P1 | Notifications : ajout, retrait, nouveau document, mention — absentes | ■ ABSENT |
| P1 | Routes API sécurisées pour toutes les opérations espace | ■ ABSENT |
| P1 | Connexion organization_members (multi-tenant) — utilise uniquement team_members | ■ PARTIEL |
| P1 | Suppression d'un espace sans confirmation — risque de perte de données | ■ PARTIEL |
| P2 | Canal de chat privé lié à l'espace (réutilise org_message_groups) | ■ ABSENT |
| P2 | Tâches liées à un espace via Productivité (space_id sur team_tasks existe) | ■ PARTIEL |
| P2 | Événements liés à un espace via Planning (space_id sur team_meetings existe) | ■ PARTIEL |
| P2 | Recherche et filtres dans la liste des espaces | ■ ABSENT |
| P2 | Archivage (is_active existe mais pas de vue filtrée actifs/archivés) | ■ PARTIEL |
| P2 | Tests de sécurité : UUID manipulé, user retiré, org croisée | ■ ABSENT |
| P3 | Densité visuelle : empty state trop grand, cartes trop espacées | ■ PARTIEL |
| P3 | Avatars membres visibles sur les cartes d'espace | ■ ABSENT |
| P3 | Toggle grille/liste + activité récente visible sur les cartes | ■ ABSENT |
| P3 | Skeletons de chargement | ■ ABSENT |

## 4 — Audit DB / RLS

### 4.1 Tables concernées

| TABLE | org_id ? | space_id ? | user/member | RLS | RISQUE |
|---|---|---|---|---|---|

| | | | | | |
|---|---|---|---|---|---|
| private_spaces | ■ NON | — | user_id (chef) | user_id = auth.uid() | P0 — aucune isolation org |
| team_members | ■ OUI | ■ space_id FK | user_id (chef) | user_id = auth.uid() | P0 — membre ne peut pas lire |
| team_tasks | ■ OUI | ■ space_id FK | user_id (chef) | user_id = auth.uid() | P1 — membres exclus |
| team_meetings | ■ OUI | ■ space_id FK | user_id (chef) | user_id = auth.uid() | P1 — membres exclus |
| team_messages | ■ OUI | ■ space_id FK | user_id (chef) | user_id = auth.uid() | P1 — membres exclus |
| space_members | ■ N/A | ■ N/A | ■ N/A | ■ TABLE ABSENTE | P0 — pas de rôles |
| space_files | ■ N/A | ■ N/A | ■ N/A | ■ TABLE ABSENTE | P1 — documents absents |
| space_activity_log | ■ N/A | ■ N/A | ■ N/A | ■ TABLE ABSENTE | P2 — audit absent |

### 4.2 Problème multi-tenant principal

La table **private_spaces** utilise **user_id** (= l'id du chef) au lieu d'**organization_id**. Cela signifie qu'un responsable avec plusieurs organisations ne peut pas isoler les espaces par organisation. De plus, si deux chefs appartiennent à la même organisation, leurs espaces ne sont pas liés à cette organisation.

```
-- Chemin de validation requis (inexistant actuellement) :
-- auth.uid() → organizations.owner_id → organization_members.user_id
-- → space_members.user_id → space_members.role → ressource
```

**■ La chaîne de validation complète n'existe pas. L'isolation repose uniquement sur user_id.**

## 5 — Audit Storage

Aucun bucket Supabase Storage n'est dédié aux Espaces Privés. Il n'existe ni table **space_files**, ni bucket **space-files**, ni signed URLs.

| | |
|---|---|
| Bucket dédié aux espaces | ■ ABSENT |
| Upload de fichiers dans un espace | ■ ABSENT |
| Signed URLs pour accès sécurisé | ■ ABSENT |
| Révocation d'accès à l'exclusion d'un membre | ■ ABSENT |
| Prévisualisation (PDF, images) | ■ ABSENT |
| Bucket non-public pour fichiers privés | ■ ABSENT |

Le bucket devra être créé avec **public: false** et toutes les URLs générées via **storage.createSignedUrl()** avec une durée limitée. Un membre retiré d'un espace doit immédiatement perdre l'accès — les URLs signées encore valides représentent une fenêtre de risque à maîtriser.

## 6 — Audit Permissions

### 6.1 Modèle cible

**Owner** — créateur de l'espace. Contrôle total : modifier, supprimer, gérer les membres et les rôles.

**Admin** — peut gérer le contenu et les membres (hors suppression de l'espace).

**Member** — collaboration complète : lire, écrire, uploader des fichiers, créer des tâches.

**Viewer** — lecture seule. Ne peut pas modifier ni uploader.

### 6.2 État actuel

| | |
|---|---|
| Rôle Owner défini et appliqué | ■ ABSENT |
| Rôle Admin défini et appliqué | ■ ABSENT |
| Rôle Member défini et appliqué | ■ ABSENT |
| Rôle Viewer défini et appliqué | ■ ABSENT |
| Vérification côté API (pas seulement frontend) | ■ ABSENT |
| RLS pour les membres non-chef | ■ ABSENT |
| Protection UUID manipulé | ■ ABSENT |

**■ Actuellement : masquer un bouton est la seule 'sécurité'. N'importe qui connaissant un UUID d'espace peut le manipuler via l'API Supabase directe.**

## 7 — Connexions ERP

| MODULE | CONNEXION | ÉTAT | NOTE |
|---|---|---|---|
| Équipe | team_members.space_id → private_spaces.id | ■ Partiel | FK existe, RLS membres manque |
| Productivité | team_tasks.space_id → private_spaces.id | ■ Partiel | FK existe, UI non connectée |
| Planning | team_meetings.space_id → private_spaces.id | ■ Partiel | FK existe, UI non connectée |
| Chat | org_message_groups (canal privé) | ■ Absent | Réutiliser canal org, pas recréer |
| Documents | space_files (table + bucket) | ■ Absent | Toute l'infrastructure manque |
| CRM | Dossiers commerciaux confidentiels | ■ Absent | Optionnel — phase 5+ |
| Contrats | Accès restreint à un espace | ■ Absent | Optionnel — phase 5+ |
| RH / Paie | Accès ultra-restreint | ■ Absent | Modèle d'autorisation explicite requis |
| Notifications | Système push DJAMA existant | ■ Absent | Réutiliser push_subscriptions |
| Activité | org_activity_log ou space_activity_log | ■ Absent | Journal dédié recommandé |

## 8 — UX / Design

### 8.1 Ce qui fonctionne bien

• Structure grille 3 colonnes responsive — bonne base

• Code d'accès masqué/révélé — pattern sécurisé visuellement

• Cartes avec barre de couleur identification — différenciation immédiate

• Bouton création GOLD avec gradient — cohérent avec la charte DJAMA

• Modal création claire avec couleur, code généré aléatoirement

### 8.2 Ce qui doit être amélioré (P3)

• **Empty state** : occupe tout l'écran, trop espacé — réduire à 60% de hauteur

• **Densité cartes** : ajouter avatars membres (5 max + compteur), activité récente

• **Pas de recherche** : dès 5+ espaces, indispensable

• **Pas de filtres** : actifs / archivés / par membre

• **Pas de skeletons** : le spinner simple est insuffisant

• **Pas de vue liste** : la grille seule est limitante sur desktop

• **Suppression directe** (Trash2) sans confirmation — risque UX

• **Propriétaire non visible** sur les cartes

### 8.3 Maquette carte enrichie (cible)

Une carte espace devrait afficher : **nom** | **couleur bar** | **description (2 lignes)** | **avatars membres** (5 max) | **compteur documents** | **compteur tâches ouvertes** | **activité récente** (dernière action) | **badge actif/archivé** | **propriétaire**

## 9 — Roadmap d'implémentation

### Phase 1 — P0 — Sécurité & architecture multi-tenant

- Ajouter organization_id sur private_spaces (migration idempotente)
- Créer table space_members (id, space_id, user_id, org_id, role, invited_by, joined_at)
- Réécrire toutes les RLS : owner via organizations, membres via space_members
- Supprimer les appels client directs — créer routes API sécurisées
- Valider : auth.uid() → org → space_members → rôle → ressource

**Résultat :** **Migration 108. Routes /api/espaces/*. RLS complète.**

### Phase 2 — P1 — Espaces + membres + rôles complets

- Routes API : POST /create, DELETE /delete, GET /list, PATCH /update
- Routes membres : POST /members/add, DELETE /members/remove, PATCH /members/role
- Refactor frontend pour utiliser les API (plus d'appels directs)
- Gestion des rôles owner/admin/member/viewer dans l'UI
- Recherche membres depuis organization_members (pas uniquement team_members)
- Confirmer avant suppression d'un espace

**Résultat :** **Toutes les opérations passent par API. Rôles visibles dans l'UI.**

### Phase 3 — P1 — Documents / fichiers / Storage sécurisé

- Créer bucket Supabase 'space-files' (public: false)
- Créer table space_files (id, space_id, org_id, name, path, size, type, uploaded_by, created_at)
- Routes API : POST /files/upload, GET /files/list, DELETE /files/delete, GET /files/url
- Signed URLs avec durée limitée (ex : 1h) — révocation à l'exclusion
- Prévisualisation PDF/images dans le panneau espace
- RLS sur space_files via space_members

**Résultat :** **Upload, download, prévisualisation sécurisés.**

### Phase 4 — P2 — Chat + activité + notifications

- Canal privé par espace via org_message_groups (is_direct=false, space_id FK)
- Ne pas recréer un troisième système de chat
- Créer table space_activity_log (space_id, org_id, user_id, action, details, created_at)
- Logguer : ajout membre, retrait, upload, suppression fichier, modification
- Notifications push (push_subscriptions) : ajout à espace, nouveau doc, mention

**Résultat :** **Chat canal dédié. Journal d'activité. Notifications.**

### Phase 5 — P2 — Connexions Productivité + Planning

- Afficher les tâches liées (team_tasks.space_id) dans le panneau espace

- Afficher les événements liés (team_meetings.space_id) dans le panneau espace
- Filtrer par espace dans Productivité et Planning
- Ne pas dupliquer les données — l'espace est une couche de permission/organisation

**Résultat :** **Tâches et événements visibles depuis l'espace sans duplication.**

### Phase 6 — P2 — Recherche + filtres + archivage

- Recherche full-text dans les espaces (nom, description, membres)
- Filtres : actifs / archivés / mes espaces / tous
- Tri : date, nom, activité, membres
- Favoris (pinned spaces)
- Pagination si > 20 espaces

**Résultat :** **Navigation fluide dans les espaces.**

### Phase 7 — P3 — Design premium + responsive

- Cartes enrichies : avatars membres, compteurs documents/tâches, activité récente
- Vue liste / grille toggle
- Skeletons de chargement
- Empty state compact et moins imposant
- Confirmation modale avant suppression
- Dark/light mode cohérent sur tous les nouveaux composants
- Responsive mobile : pile verticale, cartes full-width

**Résultat :** **Interface ERP premium, dense et professionnelle.**

### Phase 8 — Tests sécurité + performances + audit

- Org A owner tente d'accéder aux espaces d'Org B → 403/vide
- Membre A2 tente d'accéder à espace privé A1 (dont il n'est pas membre) → 403
- UUID manipulé dans l'URL → 403
- Appel API direct sans token → 401
- Membre retiré tente d'accéder au fichier Storage → signed URL expirée → 403
- Utilisateur suspendu → accès refusé
- Performances : < 200ms pour liste espaces (index OK)
- Audit final RLS avec supabase-test ou script de validation

**Résultat :** **Zéro fuite de données confirmée. Prêt pour production.**

## 10 — Risques

**R1 — CRITIQUE : Fuite de données inter-organisations**

Sans organization_id et sans RLS basée sur l'appartenance à l'organisation, un utilisateur malveillant qui devine un UUID d'espace peut potentiellement y accéder. À corriger en Phase 1 avant tout autre développement.

**R2 — ÉLEVÉ : Intégrité des données à la suppression**

La suppression d'un espace via deleteSpace() est immédiate, sans confirmation, sans vérification de droits côté serveur. Un espace avec des fichiers ou tâches liées peut être supprimé accidentellement.

**R3 — MOYEN : Liens d'invitation brisés**

Les liens /membre/login?code=... générés et copiés par les chefs pointent vers une page inexistante. Les membres invités ne peuvent pas rejoindre l'espace. À corriger en Phase 2.

**R4 — MOYEN : Migration avec données existantes**

L'ajout d'organization_id (NOT NULL) sur private_spaces nécessite un backfill soigneux via organizations.owner_id. La migration doit rester nullable initialement, être backfillée, puis contrainte. Ne jamais forcer NOT NULL sans backfill complet.

**R5 — FAIBLE : Performance à l'échelle**

Les index actuels (idx_ps_user, idx_tm_space) sont suffisants pour < 1000 espaces. Au-delà, envisager une pagination et des index composites sur (organization_id, is_active).

## 11 — Recommandations

**1. Ne pas déployer en production tant que Phase 1 n'est pas complète.**

Le module actuel n'a pas d'isolation multi-tenant. Il ne doit pas être accessible à des utilisateurs de production tant que les P0 ne sont pas corrigés.

**2. Commencer par la Phase 1 uniquement.**

Les Phases 1 et 2 sont des prérequis bloquants pour tout le reste. Ne pas travailler sur le design (Phase 7) avant d'avoir une architecture sécurisée.

**3. Réutiliser les modules existants, ne pas recréer.**

• Chat → org_message_groups (ne pas créer de troisième système)

• Tâches → team_tasks + space_id (déjà existant)

• Événements → team_meetings + space_id (déjà existant)

• Membres → organization_members (pas uniquement team_members)

**4. Toujours valider côté serveur.**

Chaque opération (créer, supprimer, ajouter un membre, uploader un fichier) doit passer par une route API Next.js qui valide l'authentification, l'appartenance à l'organisation et le rôle dans l'espace.

**5. Autorisation explicite pour RH et Contrats.**

Ne pas connecter automatiquement les données RH ou de paie à un espace privé. Chaque connexion à un module financier ou RH nécessite un modèle d'autorisation explicite et une RLS dédiée.

---